

Task Distribution

Member 1: (Data Structures & Allocation & MMU) → Magdy

Focus: Setting up the backbone of the memory system and managing free space.

1. Define Structures:

- Design the **Page Table Entry (PTE)** structure. It must contain:
 - Frame Number (Physical Address).
 - Valid Bit (Is it in memory?).
- Design the **Frame Table / Free List**: A structure (Bit Map or Linked List) to track the 32 physical frames¹.
 - Is_free
 - Process id
 - Page number (virtual number for reverse mapping)
 - Reference bit
 - Dirty bit

2. Memory Initialization:

- Implement logic to initialize memory as "All Free" at startup².
- Write helper functions: `is_memory_full()`, `get_free_frame()`, and `free_frame(int frame_index)`.

3. Process Termination Cleanup:

- Implement logic to free *all* frames associated with a process when it finishes execution so they can be reused³.

4. MMU Helper Function:

- Implement MMU Logic

5. Process Memory Constructor:

- Implement `setup_page_table(int pid)`. It allocates the frame for the page table and sets initial Valid bits to 0.

Member 2: Input Handler & Process Lifecycle & Logging → Yahia

Focus: Modifying inputs, PCB, and handling the "birth" of a process.

1. Process Generator Modification:

- Modify process_generator.c to read the new columns in processes.txt: Base (Disk Page Number) and Limit (Number of Pages)⁴.
- Pass this data to the Scheduler.

2. PCB Updates:

- Update the Process Control Block (PCB) in the Scheduler to store:
 - Disk location info (Base/Limit).

3. Startup Logic:

- When a process arrives, CALL setup_page_table(pid).
- CALL the function to load the first page.

4. Logging Generator:

- Create memory.log.
- Implement strict formatting for events: Page Faults, Allocations, and Swapping.
- *Note:* Ensure output matches the exact text format provided in the PDF to pass automatic comparison tests¹⁴.

Member 3: The Integrator (Scheduler Blocking) → Khaled

Focus: The runtime loop, handling requests, and integration with Round Robin.

1. Request Processing Loop:

- Parse the per-process memory request files (time, address, R/W)⁸.

2. Page Fault Handling:

- Check the Page Table for the requested page.
- **If Valid:** Allow access (update Reference/Dirty bits).
- **If Invalid (Page Fault):**
 - Trigger the "Block Process" mechanism⁹.
 - Move process from **Running** to **Blocked** and remove from ready queue.
 - Simulate Disk Access (Wait 10 cycles).
 - Call Member 4's logic to fetch the page/swap.

- Move process **Blocked** to **Ready** queue.

3. Round Robin Integration:

- Ensure the Round Robin scheduler respects the blocking. If a process faults, it yields the CPU immediately, waits for IO, and then returns to the queue¹⁰¹⁰¹⁰¹⁰.

Note: we need to differentiate between process state that is waiting on dependency and that waiting on IO. Divide Blocked state into BLOCKED_PAGE_FAULT and BLOCKED_DEPENDENCY. Make any needed modifications in scheduler algorithms.

Member 4: The Swapper (Second Chance Algo) → Badr

Focus: The core algorithmic complexity and output verification.

1. Replacement Algorithm (Second Chance):

- Implement the logic to scan frames.
- Check Reference Bit:
 - If 1: Set to 0 and continue.
 - If 0: Select as victim.

2. Swapping Mechanism:

- **Swap Out:** Check the **Dirty Bit** of the victim. If Dirty, write to disk (simulate extra 10 cycles). Update the victim's Page Table to "Invalid".
- **Swap In:** Load the new page into the freed frame. Update the requester's Page Table to point to this frame.